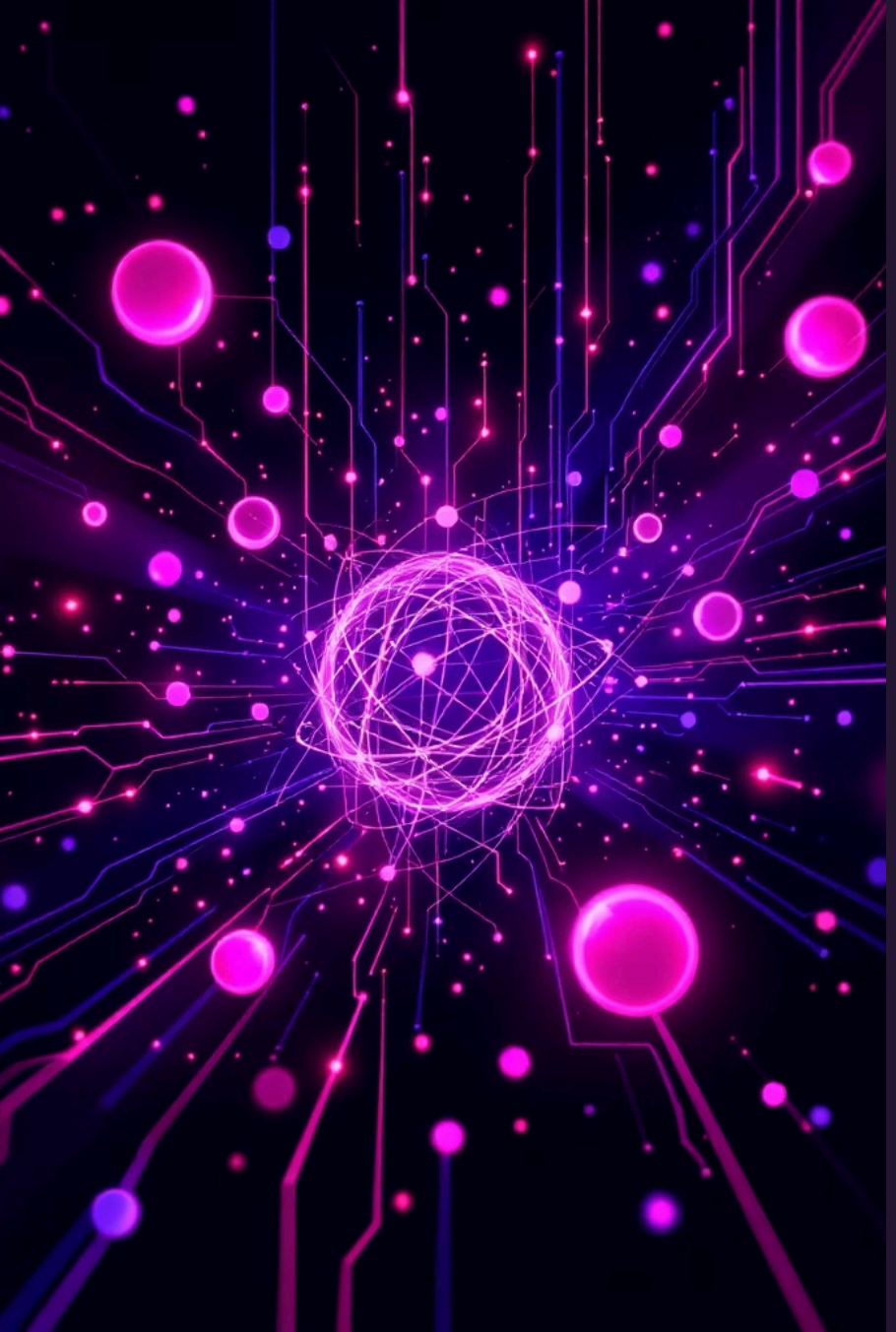


# Quantum Portfolio Optimization Using Variational Quantum Algorithms

A hybrid quantum–classical framework for financial portfolio optimization, implemented in PennyLane and validated on real quantum hardware.

KRISHNAM · ANUSHRI · KAVIN

Made with GAMMA

# Why Portfolio Optimization?

## The Classical Challenge

Portfolio optimization grows exponentially harder as the asset universe expands. Classical Markowitz mean–variance optimization faces **combinatorial complexity** and the **curse of dimensionality** for large asset sets.

## The Quantum Opportunity

Hybrid quantum–classical methods offer a compelling path forward. Our goal: demonstrate variational quantum techniques on a practical NISQ-era finance problem and rigorously evaluate their performance.

Made with GAMMA

# Classical Background: Markowitz Framework

## The Core Objective

Minimize portfolio risk while maximizing expected return:

$$\min_{w} \ \lambda w^T \Sigma w - \mu^T w$$

**w**

Portfolio weight vector — fraction of capital per asset

**μ**

Expected returns vector — historical mean returns per asset

**Σ**

Covariance matrix — captures inter-asset return correlations

**λ**

Risk-aversion parameter — governs the risk–return trade-off

## Scalability Problem

Solving this exactly for hundreds of assets demands **quadratic growth** in covariance computation and becomes intractable for combinatorial variants. A 256-asset universe is a strong stress test for any optimizer.

This scalability bottleneck is precisely what motivates a quantum approach.

Made with GAMMA

# Methodology: Data Pipeline & Quantum Model

### Feature Engineering

Compute returns, μ and Σ.

### Quantum Circuit

8-qubit PennyLane ansatz, 4 SEL.

### Data Acquisition

256 tickers, 1 year daily prices.

### VQE Optimization

Adam optimizer, 61 iterations.

The pipeline seamlessly integrates live market data with a parameterized quantum circuit, forming a closed feedback loop between quantum state measurement and classical gradient-based optimization.

Made with GAMMA

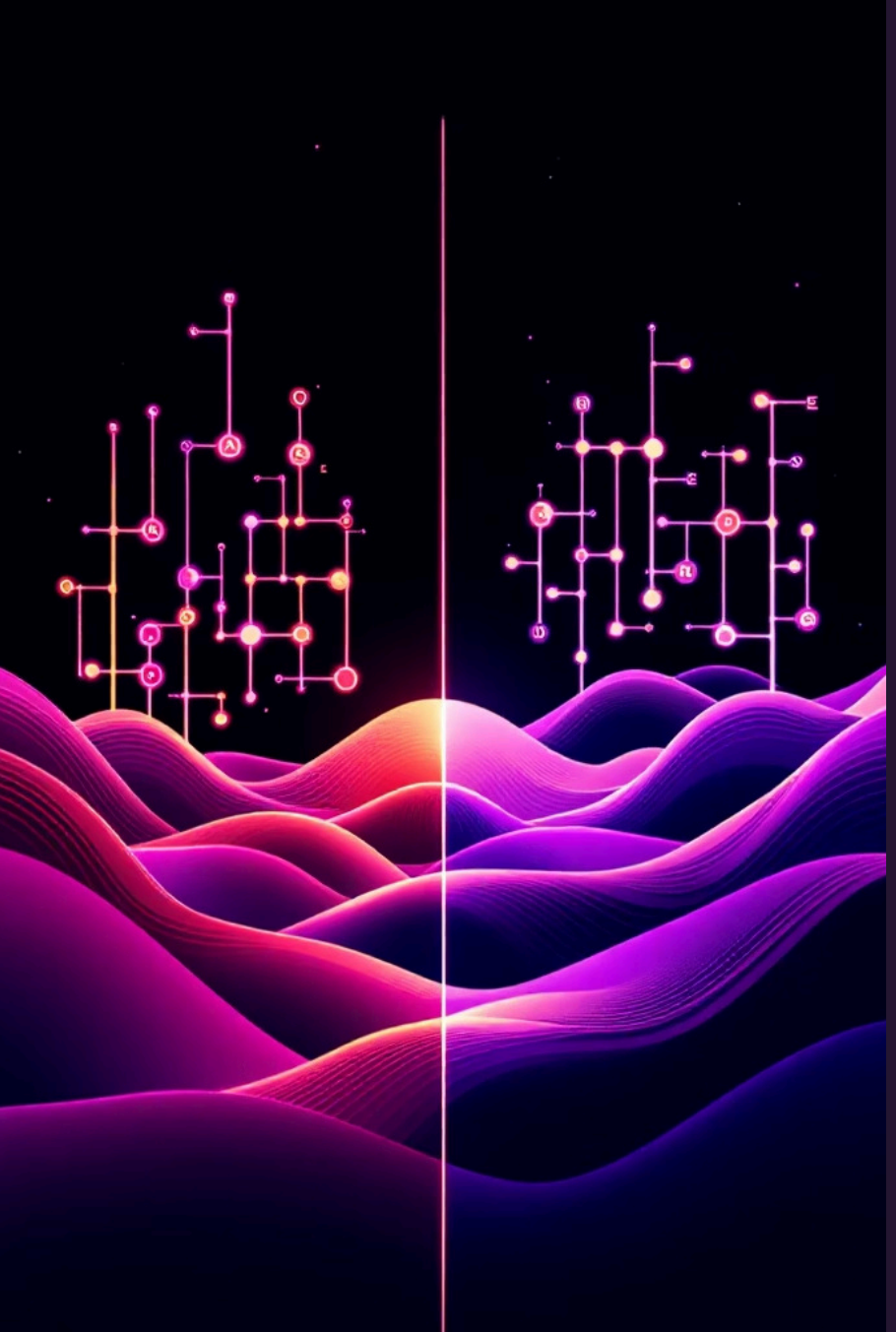
# The VQE-Style Quantum Circuit

## Circuit Architecture

- **8 qubits** compress a 256-asset allocation space
- **4 Strongly Entangling Layers** — parameterized RX, RY, RZ rotations on every qubit
- **CNOT gates** introduce entanglement to capture inter-asset correlations
- Measurement yields a probability distribution directly interpreted as normalized portfolio weights: **w = QuantumProbabilities(θ)**

Normalization is automatic — no post-processing constraint required.

Made with GAMMA



# VQE vs. QAOA: Why We Chose VQE

## QAOA – Not Suitable

Designed for **binary optimization** (include/exclude assets). Requires **~1 qubit per asset** which only tells whether to buy a stock or not but doesn't the quantity was expanded by using basis encoding and using 2 qubit but this was also limited to a range of 0 to 3.

## VQE – Our Choice

Naturally produces **normalized continuous weights** from measurement probabilities. Compresses 256 assets into just **8 qubits** — hardware-efficient and NISQ-compatible. We get the optimized eigenvalues for eigenvectors .We can 2^n number of stocks for n number of qubits.

Made with GAMMA

# Optimization & Cost Function

## Classical Cost Function

$$C(\theta) = \lambda w^T \Sigma w - \mu^T w$$

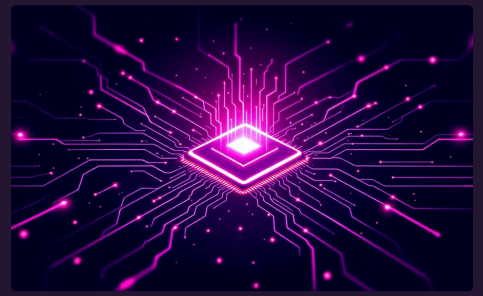
Where **w = QuantumProbabilities(θ)** — weights are derived entirely from quantum circuit output. The Adam optimizer runs for **61 iterations**, updating θ via backpropagation through the circuit.

## Post-Convergence Pipeline

- Portfolio weights ranked by magnitude
- Capital allocation computed per asset
- Risk–return statistics evaluated
- 300-day growth projected: $V(t) = V_0(1+\mu)^t$

Made with GAMMA

# Discussion: Strengths & Limitations



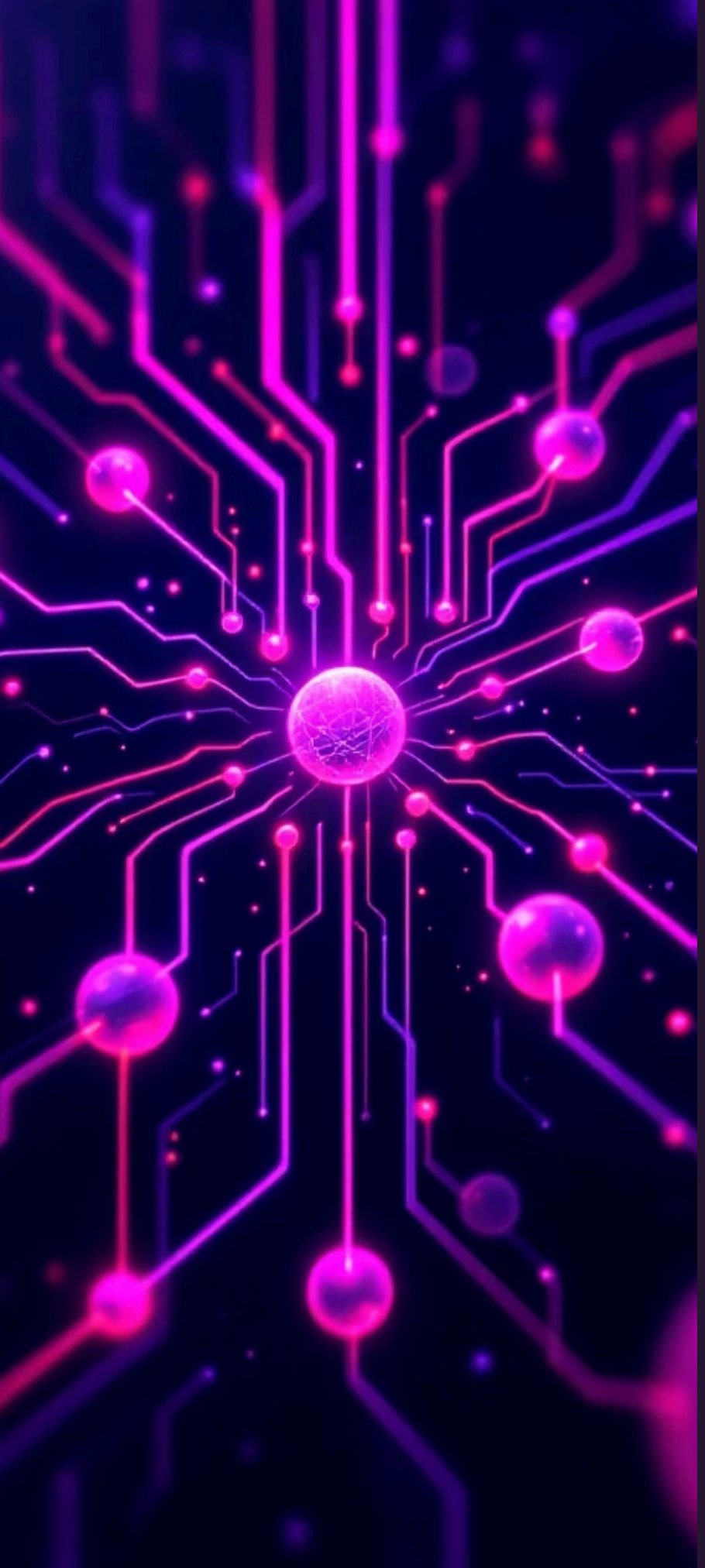
## Key Strengths

- 256 assets compressed into **8 qubits** via quantum state encoding
- Adam optimizer converges stably; normalization is automatic
- Validated on **real 27-qubit hardware** — not just simulation
- QAOA vs. VQE comparison provides rigorous benchmarking

## Current Limitations

- Classical covariance estimation remains **computationally heavy**
- Hyperparameter sensitivity (layers, λ, learning rate)
- Historical return assumptions may not predict future behavior
- Gate error and decoherence constrain circuit depth on hardware

Made with GAMMA



# End-to-End Workflow: From Data to Decision

### Data Acquisition

Fetch historical market prices for 256 assets (1 year daily).

### Feature Engineering

Calculate daily returns, mean returns (μ), and covariance matrix (Σ).

### Quantum Circuit Construction

Initialize 8-qubit PennyLane ansatz with 4 Strongly Entangling Layers.

### VQE Optimization

Adam optimizer minimizes cost function over 61 iterations.

### Portfolio Extraction

Derive optimized portfolio weights (w) from quantum probabilities.

### Results Visualization

Project 300-day growth and analyze risk-return profiles.

Our integrated pipeline seamlessly blends classical data processing with quantum computation, generating an optimized portfolio through iterative feedback between the quantum circuit and the classical optimizer.

Made with GAMMA

```python
def fetch_tickers():
    url = "https://en.wikipedia.org/wiki/List_of_S%26P_500_companies"
    headers = {'User-Agent': 'Mozilla/5.0'}
    resp = requests.get(url, headers=headers)

symbols = fetch_tickers()
raw_download = yf.download(symbols, period="1y", progress=False)

prices = raw_download.get('Adj Close', raw_download.get('Close'))

prices = prices.ffill().bfill().fillna(0)
```

# Data Acquisition & Preparation

1. Automatically scrapes the S&P 500 ticker list
2. Standardizes ticker symbols for Yahoo Finance compatibility
3. Downloads 1 year of daily historical prices
4. Handles missing or irregular data using forward/backward fill
5. Ensures the dataset always has exactly 256 assets

Made with GAMMA

```
STOCKS_COUNT = 256
QUBITS = 8
CAPITAL = 1500000
RISK_LAMBDA = 2.0
DAYS_HORIZON = 200
```

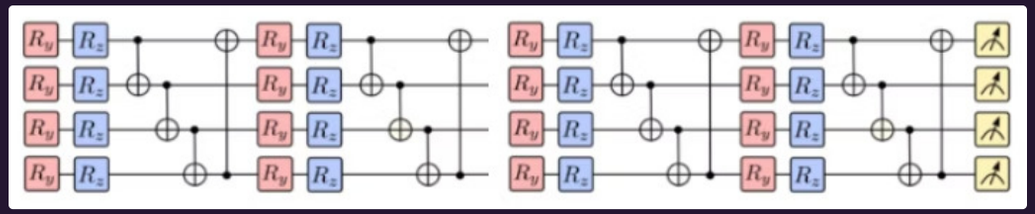
```
returns_df = prices.pct_change().fillna(0)
avg_returns = np.nan_to_num(returns_df.mean().values)
cov_matrix = np.nan_to_num(returns_df.cov().values)
individual_risks = np.sqrt(np.diag(cov_matrix))
```

# Computing Expected Returns and Risk Matrix

1. The average of daily returns is computed to estimate the expected return vector (μ) for all assets.
2. The covariance matrix is calculated to capture how different stocks move relative to each other.
3. This covariance matrix (Σ) represents the overall risk structure of the portfolio.
4. The diagonal elements of the covariance matrix are used to compute individual asset volatilities.
5. These computed quantities (μ and Σ) serve as the core inputs to the portfolio optimization objective.

Made with GAMMA

# Quantum Circuit and Objective Function Implementation



```python
dev = qml.device("default.qubit", wires=QUBITS)

@qml.qnode(dev)
def get_weights(params):
    qml.StronglyEntanglingLayers(weights=params, wires=range(QUBITS))

def objective(params):
    w = get_weights(params)
    portfolio_return = pnp.dot(w, avg_returns)
    portfolio_risk = pnp.dot(w, pnp.dot(cov_matrix, w))
    return (RISK_LAMBDA * portfolio_risk) - portfolio_return
```

```
STOCKS_COUNT = 256
QUBITS = 8
CAPITAL = 1500000
RISK_LAMBDA = 2.0
DAYS_HORIZON = 200
```

1. Initializes an eight-qubit PennyLane simulator device for portfolio modeling.
2. Uses 'n' variational layers for expressive yet shallow quantum circuit.
3. Each layer applies RX, RY, RZ parameterized single-qubit rotations.
4. CNOT gates entangle qubits to capture asset correlation effects.
5. Circuit measurement produces normalized probability-based portfolio weight distribution outputs.
6. Objective computes return and covariance-based portfolio risk tradeoff value.
7. Classical optimizer updates parameters in hybrid VQE training loop.

Made with GAMMA

```python
weights_shape = qml.StronglyEntanglingLayers.shape(n_layers=6, n_wires=QUBITS)
theta = pnp.random.random(weights_shape, requires_grad=True)
optimizer = qml.AdamOptimizer(stepsize=0.01)
```

```
Starting Quantum Optimization...
Step 0 | Score: -0.000441
Step 20 | Score: -0.001084
Step 40 | Score: -0.001465
Step 60 | Score: -0.001824
```

# Parameter Initialization and Optimizer Setup

1. Initializes trainable parameters randomly to start quantum optimization process.
2. Enables gradient tracking for parameters using PennyLane NumPy interface.
3. Adam optimizer configured with small stepsize for stable convergence.

Made with GAMMA

# Quantum Portfolio Extraction and Capital Allocation

```
-------------------------------------------------------
1    | CIEN     |  11.1546% | INR  167319.55
2    | GLW      |  10.3429% | INR  155143.95
3    | DG       |   4.4470% | INR   66705.21
4    | HII      |   2.5677% | INR   38516.03
5    | CNP      |   2.2711% | INR   34067.24
6    | AKAM     |   1.5633% | INR   23449.52
7    | EG       |   1.4927% | INR   22391.02
8    | AMAT     |   1.4576% | INR   21864.73
9    | CAT      |   1.1746% | INR   17619.66
10   | CBRE     |   1.1690% | INR   17535.00
```

```python
final_w = get_weights(theta)
sorted_indices = np.argsort(final_w)[::-1]
for rank, i in enumerate(sorted_indices, 1):
print(f"{rank:<5} | {name:<10} | {weight*100:8.4f}% | INR {weight * CAPITAL:10.2f}")
```

1. Extracts optimized portfolio weights from trained quantum variational circuit.
2. Sorts assets by descending probability to prioritize capital allocation.
3. Converts normalized weights into real monetary investment amounts.

Made with GAMMA

# Portfolio Visualization and Growth Forecast

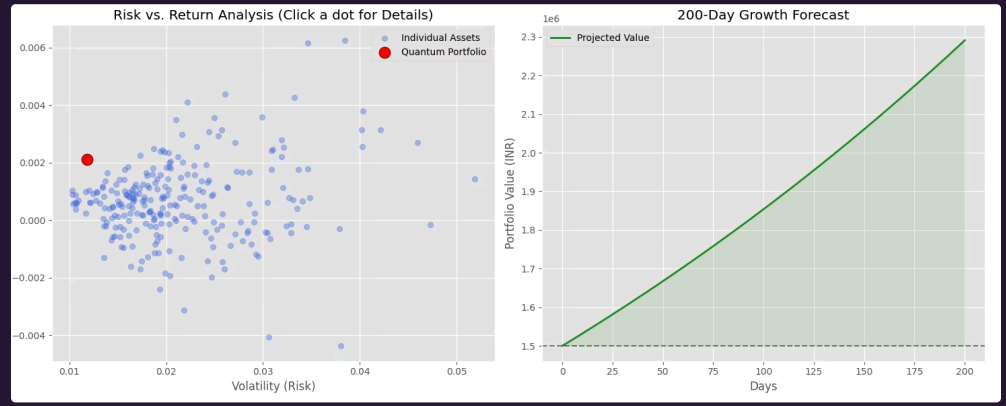
```python
import matplotlib.pyplot as plt

daily_mu = np.dot(final_w, avg_returns)
portfolio_volatility = np.sqrt(np.dot(final_w, np.dot(cov_matrix, final_w)))
forecast_values = CAPITAL * ((1 + daily_mu) ** days_range)

ax1.scatter(individual_risks, avg_returns, alpha=0.4, label='Individual Assets')
ax1.scatter(portfolio_volatility, daily_mu, color='red', label='Quantum Portfolio')

ax2.plot(days_range, forecast_values, linewidth=2, label='Projected Value')

plt.show()
```

$$FV = PV \times (1 + r_{daily})^{N_{days}}$$

1. Shows **compound growth** of portfolio value over future time horizon.
2. Graph visualizes exponential increase due to daily return compounding.
3. Risk-return scatter plot highlights optimized portfolio against individual assets.
4. Interactive graph enables deeper inspection of portfolio performance dynamics.

Made with GAMMA

# Graphs




1. Each blue dot corresponds to one individual stock in the dataset.
2. The x-axis (Volatility) shows that stock's risk (standard deviation).
3. The y-axis (Expected Daily Return) shows its average return.
4. Together, the cloud shows the risk–return landscape of all assets.
5. The red dot represents the **optimized quantum portfolio**. It shows the combined risk and return of the whole portfolio.

Made with GAMMA

# IBM Qiskit

```
qc = QuantumCircuit(QUBITS)
qc.rz(params[param_idx], i)
qc.ry(params[param_idx + 1], i)
qc.rz(params[param_idx + 2], i)
qc.cx(i, target)
backend = service.least_busy(operational=True, simulator=False)
job = sampler.run(pubs=[pub])
```

1. This code runs the optimized quantum portfolio on real IBM hardware and analyzes the results.
2. It reconstructs the strongly entangling circuit in Qiskit using layered RZ–RY–RZ rotations and CNOT entanglement, then binds trained parameters.
3. Using the IBM Quantum API token, it connects to the least-busy backend and executes it with Sampler to obtain measurement counts.
4. The top bitstrings are mapped to portfolio weights and capital allocation.

Made with GAMMA

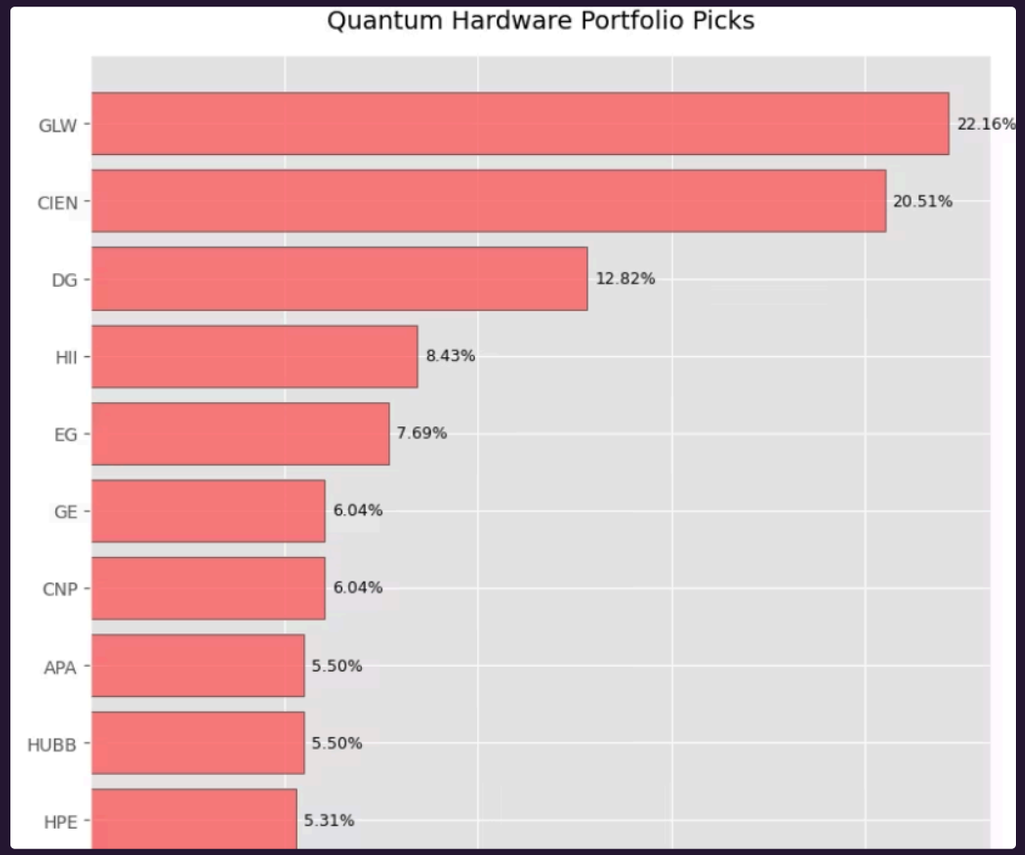
# Hardware Execution Results

## Risk-Return Performance

The VQE circuit was executed on both an **ideal simulator** and a **27-qubit quantum processor**. On the simulator, the quantum portfolio achieves high expected daily return with minimal risk, navigating the Efficient Frontier of 256 assets.

On hardware, the risk–return position remains **stable under NISQ-era noise**, with only a slight shift toward higher volatility — confirming the method's robustness against decoherence.

## Top Asset Allocation (Hardware)

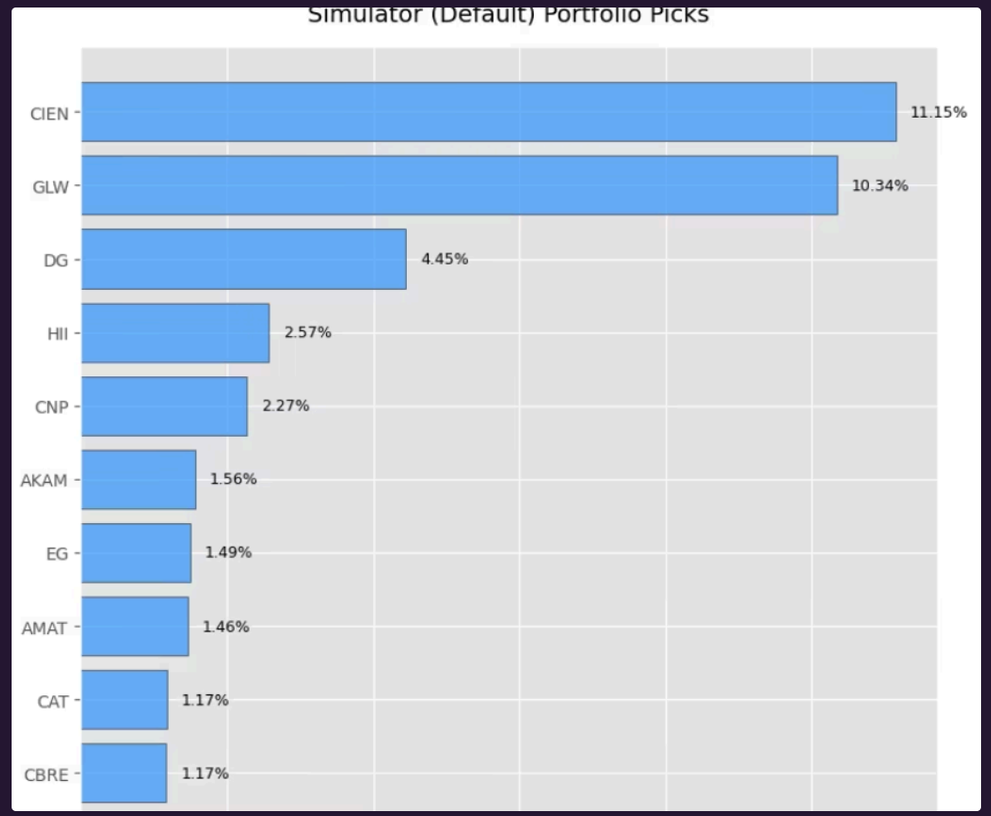



Hardware noise introduces a distributed baseline across remaining assets, but the primary allocation signal remains strong enough for practical capital deployment.

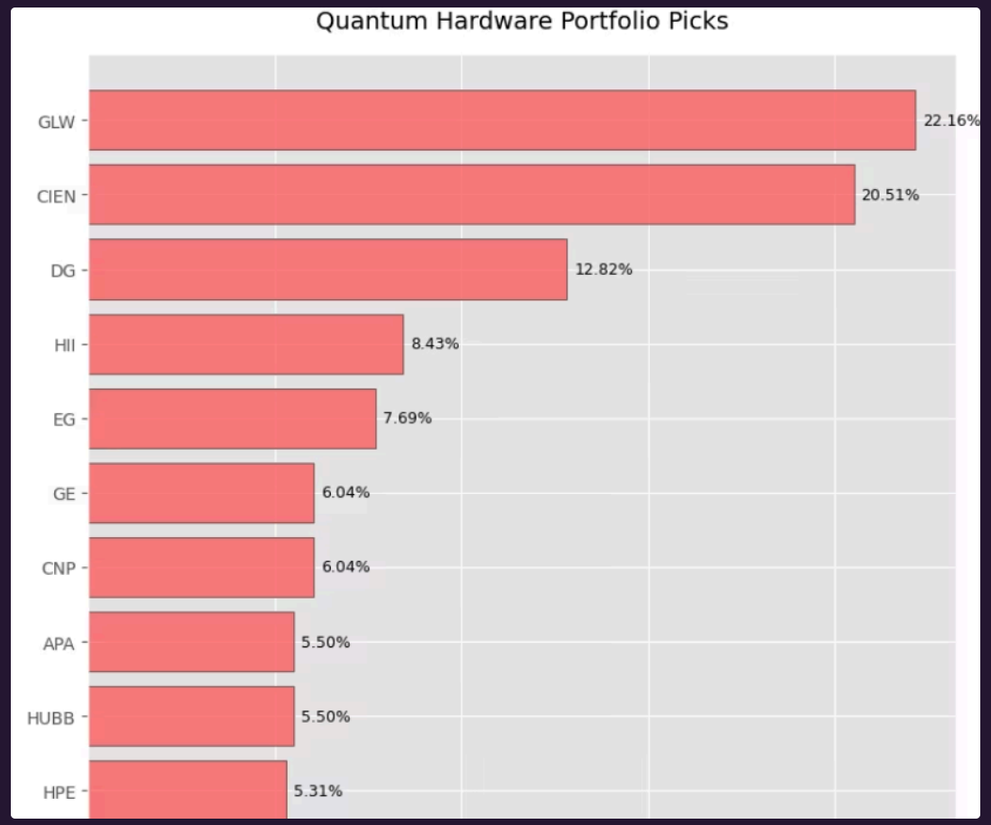
Made with GAMMA

# Simulator vs. Hardware: Performance Comparison

## Simulator Portfolio Picks




## Quantum Hardware Portfolio Picks




The charts above highlight the key differences in portfolio allocation between an ideal simulator and the noisy quantum hardware. The simulator output shows a highly concentrated portfolio, leveraging the full precision of the quantum algorithm to identify optimal assets. In contrast, the hardware execution, while still identifying the primary assets, exhibits a more distributed baseline across additional assets, reflecting the impact of NISQ-era noise. This comparison demonstrates the robustness of the VQE approach, maintaining a viable investment strategy even under present hardware constraints.

Made with GAMMA

# Conclusion & Future Directions

This project demonstrates that **variational quantum algorithms can scale to 256-asset portfolios using only 8 qubits**, achieving competitive risk–return profiles on real quantum hardware. The VQE framework outperforms QAOA for continuous weight allocation in noisy environments.

### ✓ Proven

Quantum state compression is viable for high-dimensional financial data on NISQ hardware

### → Next Steps

Reduce classical covariance overhead; explore noise-aware ansatz design and error mitigation

Link:**https://github.com/krishnamiiti29/quantum-portfolio-optimizer**

Made with GAMMA